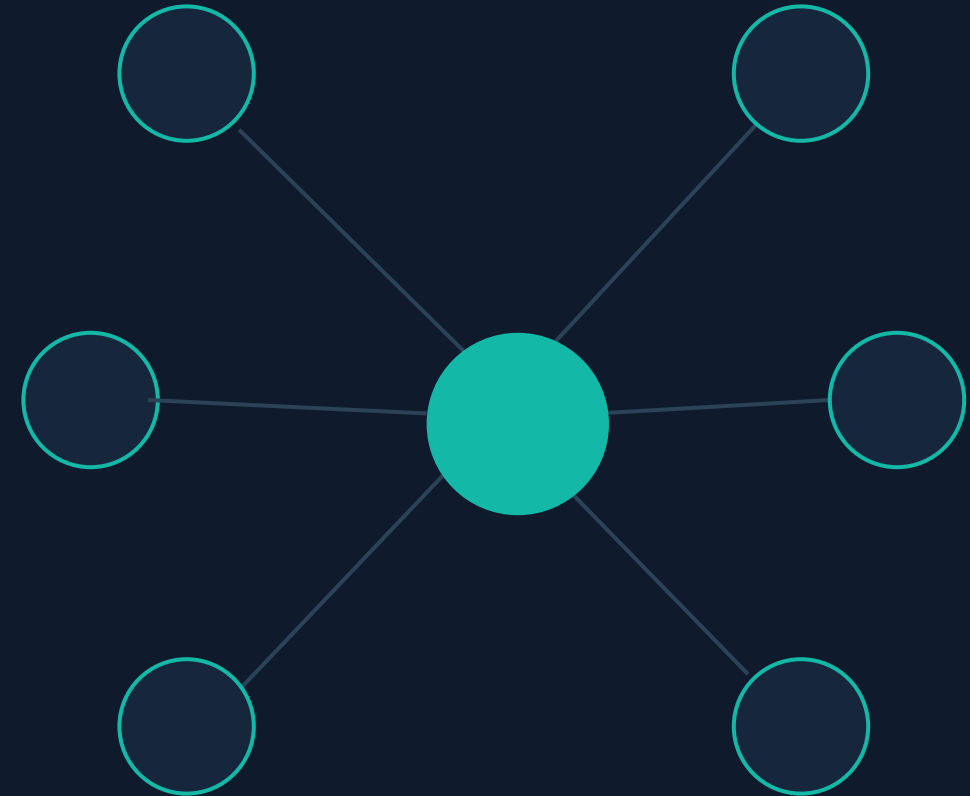


HANDS-ON LAB · INDUSTRIAL IIoT

Building an MQTT Pub/Sub Network

From your own broker to the factory floor

Lab 1 on the road to OPC UA
Mechatronics & Automation Engineering



TODAY'S MISSION

Publish one secret, collect twenty

Every student runs their own broker. You publish one secret message, then collect the secret of all 20 classmates across the Tailscale network.

By the end, your laptop is one “factory cell.” Later it hosts 10–20 PLCs instead of one secret.

AGENDA

- 1 Pub/sub & MQTT concepts
- 2 Stand up your own broker
- 3 Publish your secret (retained)
- 4 Subscribe & collect all peers
- 5 Live Streamlit dashboard
- 6 Run the game · troubleshoot
- 7 Teaser: Sparkplug B & OPC UA

1

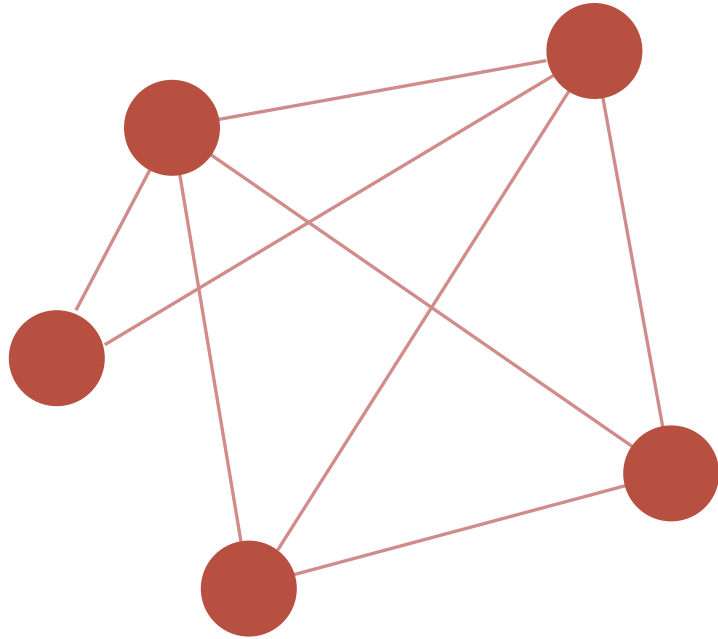
SECTION 1

Pub/Sub & MQTT concepts

Why message brokers beat point-to-point wiring

THE PROBLEM

Point-to-point doesn't scale

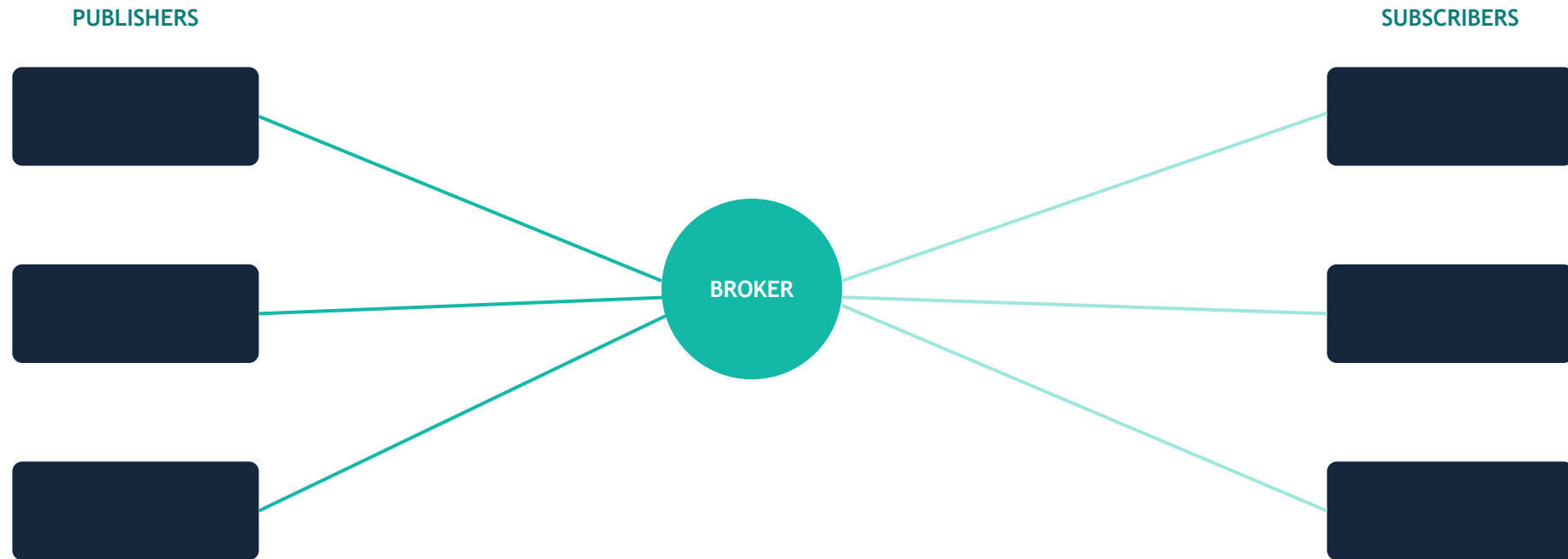


N devices → up to N^2 links

- Every device must know every other device.
- Add one machine → rewire many connections.
- Tight coupling: a change anywhere ripples everywhere.
- Impossible to maintain on a real factory floor.

THE SOLUTION

Publish / Subscribe via a broker



Nobody knows about anybody else — they only know topics. This is decoupling in space, time, and synchronization.

The MQTT vocabulary

Broker The server that routes all messages between clients.

Topic A hierarchical address, e.g. classroom/s07/secret.

Subscribe Ask the broker for all messages on a topic.

Retained Broker stores the last message for late joiners.

Wildcards + = one level, # = all levels below.

Client Any device that publishes and/or subscribes.

Publish Send a message to a topic.

QoS 0/1/2 Delivery guarantee — we use QoS 1 (at least once).

LWT Last Will — auto 'offline' message if a client drops.

Why automation engineers use MQTT

1

Lightweight

Runs on constrained PLC gateways and tiny sensors.

2

IIoT standard

The de-facto messaging layer across industrial IoT.

3

Loose coupling

Add or remove machines without rewiring the plant.

4

On-ramp to OPC UA

Same pub/sub intuition; OPC UA adds typed models.

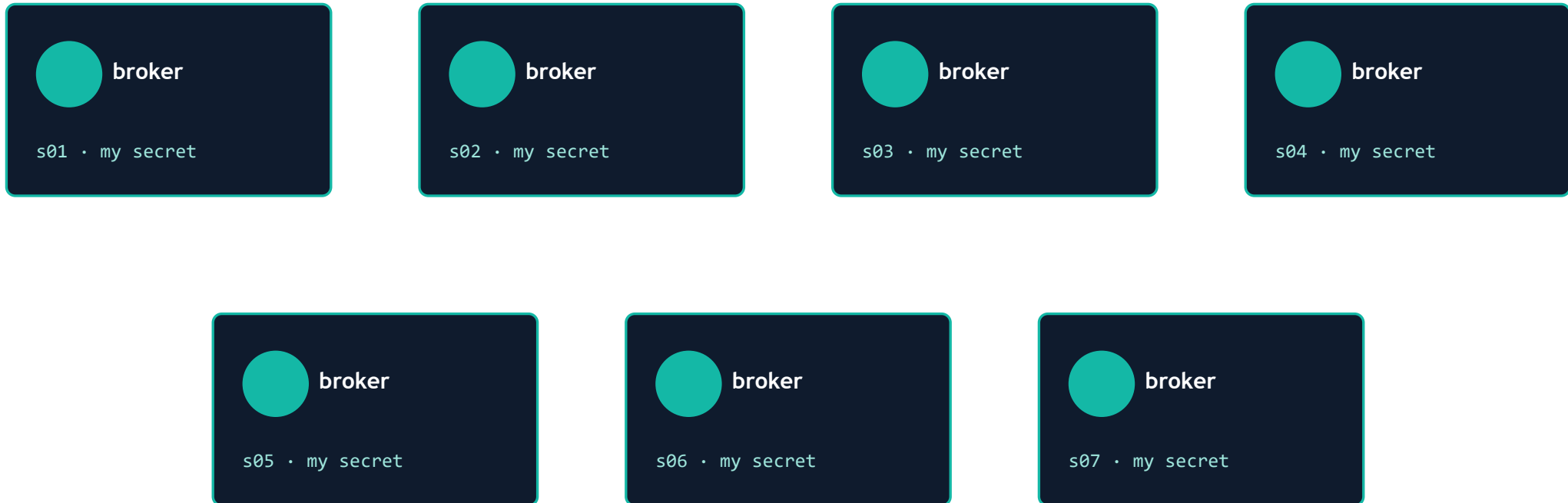
Today's "secret message" is a stand-in for a PLC tag value.

2 SECTION 2

Today's architecture

Each student is one broker — one factory cell

Each student = one broker



... 21 laptops, all on the same Tailscale tailnet

Your secret lives on YOUR broker. To read someone else's, you connect to THEIR broker.

How collection works



Feel the pain

20 manual connections is tedious — remember this. The factory fix is broker bridging (end of deck).

Conventions — everyone uses the same format

Topic structure (same on every broker)

```
...  
classroom/<id>/secret  
    # retained, QoS 1 ← the secret  
classroom/<id>/status  
    # LWT online/offline  
classroom/<id>/info
```

Payload — JSON, identical keys for everyone

```
...  
{ "id": "s07",  
  "name": "Somchai",  
  "secret": "the blue cat...",  
  "ts": "2026-06-23T10:15:00Z" }
```

Shared peer list — peers.csv

```
...  
id,name,tailscale_ip  
s01,Anan,100.101.0.1  
s02,Bee,100.101.0.2  
s07,Somchai,100.101.0.7
```

Fixed settings

- Port 1883
- QoS 1, retain = true for secrets
- IDs = seat numbers s01...s21 (never free text)
- Unique client_id: <id>-pub, <id>-sub, <id>-collector

3

SECTION 3

Stand up your broker

Install & configure Mosquitto on Windows

STEP 1

Install Mosquitto on Windows

- 1 Download the installer from mosquitto.org
- 2 Run it → accept all defaults
- 3 It installs as a Windows Service (auto-start)
- 4 We will run it from a console instead — to watch logs live

WHY CONSOLE, NOT SERVICE?

Students can see every connection and message in real time — invaluable for debugging. A silent background service hides all of that and can grab port 1883 first.

STEP 2

Stop the auto service

- 1 Open services.msc
- 2 Find “Mosquitto Broker” → Stop
- 3 Set startup type to Manual for the lab

Why: a service silently running on port 1883 will conflict with the console instance you're about to start (“address already in use”).

STEP 3

The config file – mosquitto.conf

```
...  
# mosquitto.conf (lab – open network, no TLS)  
listener 1883 0.0.0.0  
allow_anonymous true
```

listener 1883 0.0.0.0

→ accept connections from the network, not just this PC

allow_anonymous true

→ no username/password (fine for a closed class net)

⚠ The #1 thing that bites people

Mosquitto 2.0 defaults `allow_anonymous` to `FALSE` and binds to `localhost` only. Without these two lines, no peer can ever connect to your broker.

STEP 4

Run the broker & open the firewall

Run in a console (leave open all session)

```
...  
cd "C:\Program Files\mosquitto"  
.\mosquitto.exe -c .\mosquitto.conf -v
```

-v = verbose: you'll watch connections appear live.

WINDOWS FIREWALL

- First run pops a dialog → Allow on Private networks.
- Or pre-create an inbound rule for TCP 1883.
- Tailscale traffic still needs the local firewall to permit 1883.

STEP 5

Smoke-test with the CLI

Terminal A — subscribe

```
...  
mosquitto_sub -h localhost -t test -v
```

Terminal B — publish

```
...  
mosquitto_pub -h localhost -t test -m "hello"
```

See “hello” appear in Terminal A → your broker works. You're ready to write Python clients.

4

SECTION 4

Python setup

Environment, paho-mqtt, and the v2 gotcha

STEP 6

Create a Python environment

```
python -m venv venv
.\venv\Scripts\activate

pip install paho-mqtt streamlit \
            streamlit-autorefresh pandas
```

Four packages: the MQTT client, the dashboard, its auto-refresh helper, and pandas for the table.

paho-mqtt 2.x changed the API

```
...  
import paho.mqtt.client as mqtt  
c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,  
                client_id="s07-pub")
```

Tell your AI assistant about this

Most tutorials online use the old v1 callback signature and will throw errors with paho 2.x. The new signature is: `on_connect(client, userdata, flags, reason_code, properties)`. If your AI helper writes v1-style code, paste this slide.

5

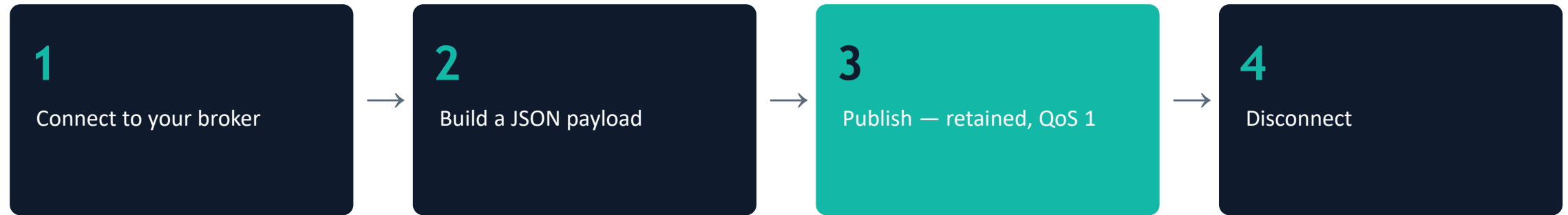
SECTION 5

Publish your secret

Build JSON, publish it retained

CONCEPT

Anatomy of a publisher



The whole publisher is ~12 lines of Python. Let's write it.

CODE

publish_secret.py

```
...  
import json, time, paho.mqtt.client as mqtt  
  
MY_ID, MY_NAME = "s07", "Somchai"  
SECRET = "the blue cat sleeps at noon"  
  
payload = json.dumps({  
    "id": MY_ID, "name": MY_NAME,  
    "secret": SECRET,  
    "ts": time.strftime("%Y-%m-%dT%H:%M:%SZ",  
                        time.gmtime())})  
  
c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,  
                client_id=f"{MY_ID}-pub")  
c.connect("localhost", 1883, 60)  
c.publish(f"classroom/{MY_ID}/secret", payload,  
         qos=1, retain=True)  
c.disconnect()
```

THE KEY LINE

retain=True

tells the broker to store your secret and hand it to anyone who subscribes later.

qos=1

= at-least-once delivery.

Why retained, and how to verify

Why retain matters here

21 students won't publish at the same instant. With retain, a classmate who connects 10 minutes later still receives your secret immediately. Without it: timing chaos and missed messages.

Verify your own retained secret

```
...  
mosquitto_sub -h localhost \  
-t "classroom/#" -v
```

You should immediately see your secret echoed back (Ctrl-C to exit). # = subscribe to everything under classroom.

6 SECTION 6

Subscribe & collect

Wildcards, then collect from all 20 peers

CODE

A simple subscriber (one broker)

```
...  
  
import paho.mqtt.client as mqtt  
  
def on_connect(client, userdata, flags,  
               reason_code, properties):  
    client.subscribe("classroom/+/secret", qos=1)  
  
def on_message(client, userdata, msg):  
    print(msg.topic, msg.payload.decode())  
  
c = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2,  
                client_id="s07-sub")  
c.on_connect = on_connect  
c.on_message = on_message  
c.connect("localhost", 1883, 60)  
c.loop_forever()
```

WILDCARDS

classroom/+/secret

one level — every student's secret

classroom/#

everything under classroom

THE CHALLENGE

Your secret is local — peers' secrets are remote

- 1 Your secret sits on YOUR broker. Everyone else's sits on THEIRS.
- 2 A normal subscriber only sees one broker at a time.
- 3 So you must connect outward to all 20 peer brokers in turn.
- 4 Read each peer's IP from the shared peers.csv — enter the collector.

collect_secrets.py (1 of 2)

Set up a local SQLite store + save helper:

```
...  
  
import csv, json, sqlite3, paho.mqtt.client as mqtt  
DB = "secrets.db"  
  
def init_db():  
    con = sqlite3.connect(DB)  
    con.execute("""CREATE TABLE IF NOT EXISTS secrets(  
        id TEXT PRIMARY KEY, name TEXT, secret TEXT,  
        src_ip TEXT, ts TEXT)""")  
    con.commit(); con.close()  
  
def save(rec, ip):  
    con = sqlite3.connect(DB)  
    con.execute("REPLACE INTO secrets VALUES (?, ?, ?, ?, ?)",  
        (rec["id"], rec.get("name", ""), rec["secret"],  
        ip, rec.get("ts", "")))  
    con.commit(); con.close()
```

collect_secrets.py (2 of 2)

Loop over peers, grab each retained secret:

```
...  
  
def on_message(client, userdata, msg):  
    rec = json.loads(msg.payload.decode())  
    save(rec, userdata["ip"])  
  
def grab_from(ip):  
    c = mqtt.Client(...VERSION2,  
        client_id=f"collector-{ip}", userdata={"ip":ip})  
    c.on_message = on_message  
    c.connect(ip, 1883, 10)  
    c.subscribe("classroom/+/secret", qos=1)  
    c.loop_start(); time.sleep(2) # let retained arrive  
    c.loop_stop(); c.disconnect()  
  
init_db()  
for row in csv.DictReader(open("peers.csv")):  
    try: grab_from(row["tailscale_ip"])  
    except Exception as e: print("skip", e)
```

TEACHING POINTS

try/except per peer

one offline classmate won't kill the run.

REPLACE INTO

idempotent — re-running just updates rows.

sleep(2)

give retained messages time to arrive.

Collect, count, and why we decouple

```
...  
python collect_secrets.py  
# up to 20 rows land in secrets.db  
python -c "import sqlite3; print(...count...)"
```

Why split the collector from the dashboard?

The collector WRITES to secrets.db. The dashboard only READS it. This sidesteps the classic Streamlit + paho threading trap (MQTT callbacks run on a non-UI thread and can't safely touch Streamlit) — and it's exactly how SCADA separates data acquisition from visualization.

7

SECTION 7

Live dashboard

Visualize every secret with Streamlit

GOAL

What we're building



Classroom Secret Collector

18 / 20

Secrets collected

id	name	secret
s01	Anan	follow the river east
s02	Bee	gear ratio is 3 to 1
s05	Chai	torque at sunrise

Auto-refreshes every 3 s · table + counter + progress

CODE

dashboard.py

```
...  
  
import sqlite3, pandas as pd, streamlit as st  
from streamlit_autorefresh import st_autorefresh  
  
st.set_page_config(layout="wide")  
st_autorefresh(interval=3000, key="r") #3s  
  
st.title("🔒 Classroom Secret Collector")  
  
df = pd.read_sql("SELECT id,name,secret,src_ip,ts"  
                " FROM secrets ORDER BY id",  
                sqlite3.connect("secrets.db"))  
  
st.metric("Secrets collected", f"{len(df)} / 20")  
st.progress(min(len(df)/20, 1.0))  
st.dataframe(df, use_container_width=True)
```

RUN IT

```
...  
  
streamlit run dashboard.py
```

Opens at localhost:8501. Re-run the collector → the numbers climb toward 20.

BONUS

Last Will – detect who's offline

```
...  
# in your publisher / a presence client:  
c.will_set(f"classroom/{MY_ID}/status", "offline",  
          qos=1, retain=True)  
c.connect("localhost", 1883, 60)  
c.publish(f"classroom/{MY_ID}/status", "online",  
          qos=1, retain=True)
```

In the factory, this is exactly how you know a cell or PLC dropped offline — the broker publishes the will automatically.

8 SECTION 8

Run the game & debug

Race to 20/20 — and fix what breaks

Class run — step by step

- 1 Everyone starts their broker in a console
- 2 Everyone publishes their secret (retained)
- 3 Confirm peers.csv has all 21 Tailscale IPs filled in
- 4 Everyone runs the collector
- 5 Open the dashboard — race to 20 / 20

Troubleshooting cheat-sheet

Symptom	Likely cause & fix
Can't connect to a peer	Their broker isn't running · firewall blocks 1883 · wrong Tailscale IP
0 secrets collected	Forgot retain=True, or wrong topic name
“Connection refused” on own broker	Service vs console conflict — port 1883 already in use
paho callback error	Missing mqtt.CallbackAPIVersion.VERSION2
allow_anonymous / auth error	Mosquitto 2.0 default is false — check mosquitto.conf

Verify visually, then confirm success

MQTT Explorer (optional)

Point the free MQTT Explorer app at a peer's Tailscale IP to SEE the whole topic tree visually. The fastest way to confirm a secret is really retained on a broker.

✓ What good looks like

- Dashboard shows 20 / 20
- Table lists every classmate
- You can explain topic, retain & wildcard

9 SECTION 9

Recap & what's next

From secrets to PLCs to OPC UA

RECAP

You built the full MQTT loop

- ✓ **Ran a broker** Mosquitto on your own machine, open to the tailnet
- ✓ **Published retained JSON** Your secret, stored for late joiners
- ✓ **Subscribed with wildcards** classroom/+ /secret across peers
- ✓ **Collected across the network** 20 brokers → one SQLite store
- ✓ **Visualized it live** Streamlit dashboard, auto-refreshing

From “secret” to PLC tag

Same pattern — just swap the payload:

```
...  
# today  
"secret": "blue cat..."
```



```
...  
# next: a factory cell  
"temp": 72.4, "rpm": 1480
```

Your laptop now hosts many tags instead of one secret. That's a factory cell — and you already know how to publish, retain, and collect it.

The scaling problem → bridging & Sparkplug B

Broker bridging

20 manual connections don't scale. A parent “plant” broker bridges all the cell brokers and aggregates them automatically.

Sparkplug B

The industrial MQTT spec: standard payloads + birth/death certificates so you always know a PLC's true state.

ISA-95 namespaces

Structured topics: enterprise / site / area / line / cell / ... so aggregation stays clean at plant scale.

NEXT LAB

Next stop: OPC UA

Where MQTT is topics + bytes, OPC UA adds a typed information model, an address space, and built-in security.

MQTT (today)

Topics · retained bytes · lightweight · pub/sub

OPC UA (next)

Typed model · address space · security · interoperability

APPENDIX

Your file checklist

mosquitto.conf

broker config (listener + allow_anonymous)

peers.csv

shared list of all 21 Tailscale IPs

publish_secret.py

publish your secret, retained

collect_secrets.py

loop peers → secrets.db

dashboard.py

live Streamlit view

Every student leaves with the same 5 files and the same conventions — that's what guarantees they interconnect.